

ROP for the Web — Compression Dictionary

Transport Research Labs

ROP for the Web — Compression Dictionary Transport Research Labs - Lenin Alevski, GitHub.

- Published: date not stated
- Original: <<https://github.com/Alevsk/compression-dictionary-transport/tree/1938adc4faf858a2e61ee9232a35afe028cd726c>>
- Preserved from: <https://github.com/Alevsk/compression-dictionary-transport/tree/1938adc4faf858a2e61ee9232a35afe028cd726c> (live) on 2026-09-13
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source from the Web Hacking Techniques Index collections, kept so the page going offline. To read the original, follow the link above.

Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

[Branches](#) [Tags](#)

[Go to file](#)

[Code](#)

Latest commit

History

[1 Commit](#)

Folders and files

|
[deployments](#)
|
[deployments](#)
|
|
| | |

docs

|

docs

|

|

| | |

exploits

|

exploits

|

|

| | |

lab

|

lab

|

|

| | |

tests

|

tests

|

|

| | |

.dockerignore

|

.dockerignore

|

|

| | |

.gitignore

|

.gitignore

|

|

|||

.python-version

|

.python-version

|

|

|||

LICENSE

|

LICENSE

|

|

|||

Makefile

|

Makefile

|

|

|||

README.md

|

README.md

|

|

|||

pyproject.toml

|

pyproject.toml

|

|

|||

View all files

| |

Repository files navigation

ROP for the Web

Security research on Compression Dictionary Transport (CDT, [RFC 9842](#)).

CDT is the browser feature that lets a server ship a response as a Brotli or Zstandard delta against a dictionary the browser cached on an earlier visit. This repository asks what happens when the party emitting that delta is not fully trusted.

If an attacker controls the response path (a compromised or insider origin, or a CDN/edge), they can assemble a response out of byte-runs that already exist in a legitimate library the victim cached, ship it as an opaque `dcb` blob, and move it past controls that inspect response bodies. The inspector cannot decompress the blob, because the dictionary lives only in the victim's browser cache. It is never in the response, and never on the server's disk.

The idea, in one analogy

The talk is called *ROP for the Web* because the mechanism mirrors Return-Oriented Programming.

| ROP (memory corruption) | CDT dictionary gadgets | | | Chain code snippets already at known addresses | Chain byte sequences already at known dictionary offsets | | | The attacker never writes shellcode to memory | The attacker never writes malicious code to the server | | | Gadgets come from legitimate code (libc) | Gadgets come from a legitimate library (jQuery, React) | | | Only the return-address chain is attacker-controlled | Only the compressed backreference stream is attacker-controlled | |

`brotli -D dictionary payload` finds every substring of the payload that already exists in the dictionary and emits a backreference ("copy 47 bytes from offset 1203") instead of the literal text. Chain enough of those and the malicious response reduces to a handful of offset and length pairs: opaque binary, with no recognizable strings left to match.

The gadget scanner

`exploits/EXP-001/gadget_scanner.py` is a ROP-style gadget finder for JavaScript dictionaries. Point it at a real library and it reports which security-relevant byte sequences (XSS sinks, exfiltration primitives, code-execution strings) already live inside it. Give it a candidate payload and it reports how much of that payload the dictionary can assemble from backreferences alone.

```
uv run python exploits/EXP-001/gadget_scanner.py lab/static/js/jquery-3.7.1.min.js \  
  --payload exploits/EXP-001/payloads/app.v2.backdoored.js  
# or: make exp-001-scan
```

The rule it measures: a payload is invisible on the wire only if every detectable sink is either present in the dictionary as a byte-run, or decomposable against runs the dictionary does contain.

Experiments

Each experiment is a self-contained, loopback-only Docker lab with an automated runner (`make exp-NNN-run`) and a full reproduction guide in its own `README.md` .

| | Shows | Defender evaded | Verdict | | | **EXP-001** | dcb-compressed XSS, assembled from a cached jQuery or React dictionary, carries zero readable signatures on the wire yet reassembles in the browser | Response-body inspection (ModSecurity + nginx) | Confirmed | | | **EXP-002** | An insider exports real secrets (app source with a DB password, customer PII with card numbers, an API key and a private key) through a benign `/export.php` . Plaintext is blocked at 403; the identical dcb response leaves at 200 | Egress WAF (Apache + mod_security2 + OWASP CRS) and a representative DLP | Confirmed | | | **EXP-003** | The same evasion one tool-class over: the plaintext control alerts, the dcb response is silent | Network IDS (Suricata response-body rules) | Confirmed | | | **EXP-006** | Attacker-chosen candidate dictionaries turn dcb response size into an oracle that recovers a synthetic identifier one byte at a time | Compression side channel (disclosure, not evasion) | Confirmed (pii / token) | |

EXP-001 through EXP-003 run the same response-path evasion against three different inspection tools, which is the point. When both an application WAF and a network IDS miss the same response, the gap is in the protocol, not in any one product. EXP-002 is the strongest case: a WAF that genuinely enforces a response-body 403, bypassed by the encoding alone.

The talk

The full write-up is the deck [docs/ROP for the Web.pdf](#) (source: `docs/presentation.html`). A runbook for demoing the labs live is in [docs/TRAINER-RUNBOOK.md](#) , and the running notes are in [docs/RESEARCH-LOG.md](#) and [docs/BLOGPOST.md](#) .

Repository layout

```
exploits/  the experiments (EXP-001, 002, 003, 006), each with its own README
lab/      the CDT demo server + vendored real libraries used as dictionaries
deployments/ Docker Compose stacks and Dockerfiles
docs/     the deck, research log, blogpost notes, and reference material
Makefile  one prefix per experiment: exp-NNN-{up,down,logs,run,clean}
```

Running an experiment

Prerequisites: Docker (with `docker compose`), the `brotli` CLI (`brew install brotli`), `openssl` , `curl` , and `u` `v` .

```
make check      # brotli on PATH + Flask import
make exp-002-run # build the stack, run the exfil demo, print the verdict, tear down
make help       # every target, grouped per experiment (try: make help | grep exp-001)
```

Each experiment's `README.md` carries the full step-by-step, including the by-hand `curl` commands.

Responsible use (lab only)

These stacks are deliberately vulnerable and ship synthetic secrets on disk: fake passwords, a fake `sk_live_` key, a fake PEM private key, and synthetic PII with test-card numbers. They bind to `127.0.0.1` only and publish no application ports to the host. Do not deploy them anywhere reachable. Every secret and card number is a lab value.

The intent is defensive. The point is to show WAF, IDS, and DLP maintainers, and the people implementing CDT, a response-inspection blind spot so it can be closed. Nothing here was run against a production CDT deployment, and the technique needs an attacker who already controls the response path. It is not initial access, and it is not a request-side WAF bypass.

References

- RFC 9842, Compression Dictionary Transport: <https://www.rfc-editor.org/rfc/rfc9842>
- MDN, Compression dictionary transport: https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/Compression_dictionary_transport

Acknowledgements

Thanks to [@iansharkey](#) for the nudge and encouragement to start looking into this area.

License

MIT. See [LICENSE](#).

By Lenin Alevski ([@Alevsk](#)).

Archived from <https://github.com/Alevsk/compression-dictionary-transport/tree/1938adc4faf858a2e61ee9232a35afe028cd726c>. Rendered offline from the local Markdown copy.